🏋️ FitTrack Project — Deep-Dive Study Guide

This document explains **every single folder and file** in your project in deep detail. Read this top-to-bottom and you will understand exactly how every piece fits together.

🗺️ Big Picture Overview

Your project is a **Fitness Tracker web application** called **FitTrack**. It is a fully client-side (front-end only) single-page application (SPA). There is **no real back-end server** — all data lives in memory in the browser while the page is open.

Tech Stack

Technology	Purpose
Vue 3	The JavaScript UI framework — builds the page components
TypeScript	A typed version of JavaScript — helps catch bugs early
Pinia	State management — a global store that all components share
Vue Router	Handles URL navigation between "pages" without reloading
Bulma CSS	A CSS framework for ready-made styling classes
FontAwesome	Icon library (the little icons you see in the UI)
Vite	A super fast build tool and dev server

📁 Root Directory: `e:\final project-webserver programming\`

This is the top-level folder. Here is everything inside it:

```
final project-webserver programming/
├── client/                  ← The main Vue 3 app (where all your real code lives)
├── no framework/            ← A separate demo using only plain HTML + Bulma
├── docs/                    ← Documentation screenshots
├── node_modules/            ← Auto-installed packages (DO NOT TOUCH)
├── README_WALKTHROUGH.md    ← Project walkthrough with screenshots
└── et --hard c02...         ← An accidentally-created file (a git command run in wrong place)
```

📄 Root-Level Files

`README_WALKTHROUGH.md`

This is a **documentation/showcase file**. It's not code — it's a markdown document that:

- Describes the features of the app (Authentication, Dashboard, Activities, Friends, Admin)
- Embeds screenshots of each page from the `docs/screenshots/` folder
- Embeds a video recording of the full user flow
- Lists the architecture, tech stack, and deployment instructions

> ***Study tip:*** *This is the "face" of your project on GitHub. When someone visits your repo, this is what they see.*

`et --hard c02...` (the weird file)

This is an **accidental file**. At some point, someone ran a git command like `git reset --hard c0200e...` but ran it in a way that created a file instead of running git. You can ignore/delete this safely.

📁 `docs/`

```
docs/
└── screenshots/    ← PNG and WEBP images used in README_WALKTHROUGH.md
```

This folder only contains static image/video assets for documentation. There is no runnable code here.

📁 `no framework/`

```
no framework/
├── index.html    ← A standalone HTML page using Bulma CSS
└── README.md     ← Brief description
```

`no framework/index.html`

This is a **completely separate mini-project**. It is a static HTML page that demonstrates Bulma CSS components **without any JavaScript framework** at all — no Vue, no React, nothing.

What it contains:

- A **Bulma navbar** at the top with a brand, nav links, and buttons
- A **3-column layout** using Bulma's grid system (`columns` / `column`)
- A **card component** (left column)
- A **task manager interface** (center column) with:
  - Tabs (Current / Completed / Upcoming / All)
  - A text input for adding a new task
  - A checklist of tasks (done with plain `<input type="checkbox">`)
  - A notification banner (the blue/green `notification is-primary`)
  - A full **contact form** with name, username, email, subject, message, checkbox, radio buttons, and submit button
- A **table of contents sidebar** (right column) using Bulma's `menu` component

> ***Why does it exist?*** *This was probably created early in the course to learn/demo Bulma CSS before using Vue. It serves as a visual reference for Bulma components.*

📁 `client/` — The Main Vue Application

This is where **all your real application code lives**. Let's go through every file and folder.

```
client/
├── src/                    ← All your source code
│   ├── assets/             ← CSS and static assets
│   ├── components/         ← Reusable Vue components
│   ├── router/             ← URL routing configuration
```

```
│   ├── stores/              ← Global state (Pinia stores)
│   ├── views/               ← Full-page Vue components
│   ├── App.vue              ← The root/shell component
│   └── main.ts              ← The entry point (where the app starts)
├── public/                  ← Static files served as-is
├── dist/                    ← The built production output (auto-generated)
├── node_modules/            ← Installed packages (auto-generated)
├── index.html               ← The ONE real HTML file of the entire SPA
├── package.json             ← Project metadata and dependency list
├── vite.config.ts           ← Vite bundler configuration
├── tsconfig.json            ← TypeScript config (root)
├── tsconfig.app.json        ← TypeScript config (for the app source)
├── tsconfig.node.json       ← TypeScript config (for Vite/Node tools)
├── eslint.config.ts         ← ESLint linting rules
├── .prettierrc.json         ← Prettier formatting rules
├── .oxlintrc.json           ← Oxlint (fast linter) configuration
├── .editorconfig            ← Editor settings (indentation, line endings)
├── .gitignore               ← Files to exclude from git
├── .gitattributes           ← Git line-ending settings
├── env.d.ts                 ← TypeScript environment type declarations
└── README.md                ← Vite-generated project README
```

⚙️ Configuration Files (Deep Dive)

`client/index.html`

This is the **only real HTML file** in the entire app. It looks minimal:

```html
<div id="app"></div>
<script type="module" src="/src/main.ts"></script>
```

The `<div id="app">` is where Vue **mounts** (injects) the entire application. Everything you see on screen is Vue inserting HTML into this div dynamically. Vite uses this file as the starting point.

`client/package.json`

This file is the **project's identity card and instruction manual for npm**. Key sections:

Scripts — commands you can run:

Script	Command	What it does
`dev`	`npm run dev`	Starts the local dev server (hot reload)
`build`	`npm run build`	Compiles for production into `dist/`
`preview`	`npm run preview`	Previews the production build locally
`type-check`	`npm run type-check`	Runs TypeScript to check for type errors
`lint`	`npm run lint`	Runs both ESLint and Oxlint to fix code issues

`format`	`npm run format`	Runs Prettier to auto-format code

Dependencies — packages required at runtime:

Package	Version	Role
`vue`	^3.5.29	The Vue 3 framework itself
`vue-router`	^5.0.3	URL routing
`pinia`	^3.0.4	State management
`bulma`	^1.0.4	CSS framework for styles
`@fortawesome/fontawesome-free`	^7.2.0	Icons

DevDependencies — packages only needed during development (not shipped to users):

- `vite`, `@vitejs/plugin-vue` — build tooling
- `typescript`, `vue-tsc` — TypeScript compilation
- `eslint`, `oxlint`, `prettier` — code quality tools

`client/vite.config.ts`

Vite is the tool that builds and serves your app. This config does two things:

1. **Plugins**: Enables the Vue plugin (so Vite understands `.vue` files) and Vue DevTools
2. **Alias**: Sets `@` as a shortcut for `./src` — so instead of writing `../../stores/auth`, you write `@/stores/auth`. You'll see this `@/` pattern everywhere in the code.

`client/tsconfig.json` / `tsconfig.app.json` / `tsconfig.node.json`

Three TypeScript config files:

- `tsconfig.json` — The root config that **references** the other two
- `tsconfig.app.json` — TypeScript settings for your actual app source code (strict mode, Vue JSX support, etc.)
- `tsconfig.node.json` — TypeScript settings for Node.js-level files like `vite.config.ts`

This split exists because source code and build-tool code have different TypeScript requirements.

`client/.editorconfig`

Tells code editors (VS Code, etc.) to use consistent settings: spaces vs tabs, indentation size, line endings, etc. Makes sure everyone working on the project writes code that looks the same.

`client/.prettierrc.json`

Prettier is an **automatic code formatter**. This file configures how your code looks after formatting: line length, quote style, semicolons, trailing commas, etc.

`client/.oxlintrc.json`

Oxlint is an extremely fast JavaScript/TypeScript linter. This config tells it which rules to apply when checking your code for common mistakes and style issues.

`client/eslint.config.ts`

ESLint is the classic JavaScript linter. It works together with Oxlint. Combined, they catch bugs, enforce best practices, and keep code consistent. The config enables Vue-specific and TypeScript-specific rules.

`client/env.d.ts`

Tells TypeScript about environment-specific types. The key line is:

```
/// <reference types="vite/client" />
```

This makes TypeScript understand Vite-specific things like `import.meta.env` (for environment variables) and `import.meta.url`.

`client/.gitignore`

Lists files and folders that git should **not** track:

- `node_modules/` (too large, regenerated via `npm install`)
- `dist/` (generated build output)
- `.env.local`, `.env.*.local` (secret keys)
- Various IDE cache folders

🚀 The App Entry Point

`client/src/main.ts`

```
import './assets/main.css'           // 1. Load global styles
import { createApp } from 'vue'      // 2. Import Vue
import { createPinia } from 'pinia'  // 3. Import Pinia
import App from './App.vue'          // 4. Import root component
import router from './router'        // 5. Import router

const app = createApp(App)           // 6. Create the Vue application
app.use(createPinia())               // 7. Register Pinia (state management)
app.use(router)                      // 8. Register Vue Router (navigation)
app.mount('#app')                    // 9. Mount into the <div id="app"> in index.html
```

This is the very first file that runs. It initializes everything and connects Pinia stores + Vue Router to the Vue app. Without this file, nothing would work.

`client/src/App.vue`

```
<script setup lang="ts">
import { RouterView } from 'vue-router'
import NavBar from './components/NavBar.vue'
</script>
```

```
<template>
  <NavBar />         <!-- Always visible at the top of every page -->
  <main class="section">
    <div class="container">
      <RouterView />   <!-- The current page's content goes here -->
    </div>
  </main>
</template>
```

This is the **root/shell component** — the outermost wrapper around the whole app. It renders:

1. `<NavBar />` — always shown at the top (persistent navigation)
2. `<RouterView />` — a placeholder that swaps in the correct "page" component based on the current URL

Think of `App.vue` as the picture frame. The `<RouterView>` is the canvas inside that frame where different pictures (pages) are displayed.

🧭 Router

`client/src/router/index.ts`

Vue Router manages navigation in a SPA. Instead of loading a new HTML page from the server, it just swaps out which Vue component is shown in `<RouterView>`.

Route definitions:

URL Path	Component	Who can access
`/`	`HomeView.vue`	Everyone
`/login`	`LoginView.vue`	Everyone
`/register`	`RegisterView.vue`	Everyone
`/dashboard`	`DashboardView.vue`	Logged-in users only
`/activities`	`ActivitiesView.vue`	Logged-in users only
`/friends`	`FriendsView.vue`	Logged-in users only
`/admin`	`AdminView.vue`	Admin users only
`/about`	`AboutView.vue`	Everyone

Navigation Guard (the security logic):

```
router.beforeEach((to, from, next) => {
  const authStore = useAuthStore()
  const publicPages = ['/', '/login', '/register', '/about']
  const authRequired = !publicPages.includes(to.path)

  // Rule 1: If page requires login and user is NOT logged in → redirect to login
  if (authRequired && !authStore.isLoggedIn) {
```

```
    return next('/login')
  }

  // Rule 2: If going to /admin but user is NOT admin → redirect to home
  if (to.path === '/admin' && !authStore.isAdmin) {
    return next('/')
  }

  next() // Allow navigation
})
```

`router.beforeEach` runs **before every route change**. It's like a security guard at the door of each page. This is how your app enforces role-based access control.

> ***Key concept***: *`createWebHistory` means URLs look like `/dashboard` instead of `/#/dashboard`. The `import.meta.env.BASE_URL` makes it work correctly whether deployed at `/` or a sub-path.*

🗃️ Stores (Pinia State Management)

Pinia stores are **global data containers**. Any Vue component anywhere in the app can read from or write to these stores. This is how data is shared between unrelated components without passing props everywhere.

`client/src/stores/users.ts`

What it manages: The list of all users in the system.

TypeScript types defined here:

```
type Role = 'admin' | 'user' | 'trainer'   // Only 3 valid roles

interface User {
  id: number
  username: string
  email: string
  fullName: string
  password?: string   // The ? means this field is optional
  role: Role
  friends: number[]   // Array of other users' IDs
  avatar: string      // URL to avatar image (uses ui-avatars.com API)
}
```

Seed data (pre-loaded users):

ID	Username	Password	Role	Friends
1	admin	admin123	admin	[2, 3]
2	john	john123	user	[1, 3]
3	jane	jane123	trainer	[1, 2]

Functions exposed by the store:

- `getUserById(id)` → finds and returns a single user by their ID
- `addUser(user)` → creates a new user (auto-generates an ID) and returns the new ID
- `updateUser(id, updates)` → patches an existing user with partial data
- `deleteUser(id)` → removes a user from the array

> ***Important***: *`users` is a `ref<User[]>` — a reactive array. When any component modifies it, all components watching it re-render automatically.*

`client/src/stores/auth.ts`

What it manages: The currently logged-in user's session.

State:

- `currentUser` — a `ref<User | null>`. `null` means logged out. A User object means logged in.

Computed properties (auto-derived, always up-to-date):

- `isLoggedIn` → `true` if `currentUser !== null`
- `isAdmin` → `true` if current user's role is `'admin'`

Functions:

- **`login(username, password)`** — searches the users array for a matching username+password pair. If found, sets `currentUser`. Returns `{ success: true/false, message: '...' }`.
- **`register(username, email, fullName, password)`** → validates all fields (length checks, uniqueness checks), creates the new user via `usersStore.addUser()`, then automatically logs them in by setting `currentUser`. The avatar is auto-generated using `https://ui-avatars.com/api/`.
- **`logout()`** → simply sets `currentUser = null`.

> ***Notice***: *This store imports and uses `useUsersStore` internally — Pinia stores can call each other! Auth depends on Users store.*

`client/src/stores/activities.ts`

What it manages: All fitness activities across all users.

TypeScript type:

```
interface Activity {
  id: number
  userId: number    // Which user this activity belongs to
  type: string      // e.g. 'Running', 'Cycling', 'Yoga'
  duration: number  // In minutes
  distance: number  // In km (or 0 if not applicable)
  date: string      // ISO date string: 'YYYY-MM-DD'
  calories: number  // Calories burned
  notes?: string    // Optional free-text note
}
```

Seed data (pre-loaded activities):

ID	User	Type	Duration	Distance	Calories

1	john (2)	Running	30 min	5 km	350
2	john (2)	Cycling	45 min	15 km	400
3	jane (3)	Yoga	60 min	0	200

Functions:

- `getActivitiesByUser(userId)` → filters and returns only activities matching the given user ID
- `addActivity(activity)` → adds a new activity with auto-incremented ID
- `updateActivity(id, updates)` → patches an existing activity
- `deleteActivity(id)` → removes an activity

`client/src/stores/counter.ts`

A leftover **example store** generated by the Vue project scaffold. It's not used anywhere in the actual app. Just a Vite starter template artifact.

🧩 Components

`client/src/components/NavBar.vue`

The **navigation bar** that appears at the top of every page (because it's placed directly in `App.vue`).

Script logic:

```
const authStore = useAuthStore()
const router = useRouter()

const logout = () => {
  authStore.logout()   // Clear currentUser in Pinia
  router.push('/login')  // Redirect to login page
}
```

Template structure:

- **Brand logo** (left): Vue logo SVG + "FitTrack" text, links to `/`
- **Navbar start** (center-left): Navigation links — shown conditionally:
  - "Home" → always visible
  - "Dashboard" → only if `authStore.isLoggedIn`
  - "Activities" → only if logged in
  - "Friends" → only if logged in
  - "Admin" → only if `authStore.currentUser?.role === 'admin'`
- **Navbar end** (right):
  - If logged in: A **dropdown** showing the user's full name and username, with a Logout button
  - If not logged in: "Login" and "Sign Up" buttons

The `v-if` directive is what hides/shows nav items based on auth state — when `currentUser` changes in Pinia, this component automatically re-renders.

Scoped styles: The `<style scoped>` section adds hover effects and an active-link underline using `border-bottom`.

📄 Views (Pages)

`client/src/views/HomeView.vue`

The **landing page** at `/`. Completely static — no script logic at all. Just a Bulma `box` with a welcome message and a bulleted list of features. Anyone can visit this page (it's in the `publicPages` list in the router).

`client/src/views/LoginView.vue`

The **login page** at `/login`.

Reactive state (refs):

- `username`, `password` — bound to form inputs via `v-model`
- `error` — shows an error message if login fails
- `loading` — toggles the button's loading spinner class

`handleLogin()` function:

1. Clears any previous error
2. Validates that both fields are filled
3. Calls `authStore.login(username, password)`
4. On success: redirects to `/dashboard` via `router.push()`
5. On failure: shows the error message

`demoLogin()` function: A shortcut for testing — clicking "Admin", "John", or "Jane" buttons pre-fills the credentials and calls `handleLogin()` automatically. Very useful for demos.

Template: A centered Bulma card with username/password inputs, a Login button, demo account buttons, and a link to the register page.

`client/src/views/RegisterView.vue`

The **registration page** at `/register`.

Reactive state:

- `fullName`, `email`, `username`, `password`, `passwordConfirm` — form fields
- `error`, `success` — feedback messages
- `loading` — button spinner

`register()` function:

1. Checks `password === passwordConfirm` (client-side validation)
2. Calls `authStore.register(...)` which validates username length (≥3), password length (≥6), and uniqueness of username and email
3. On success: shows a success message for 1 second, then redirects to `/dashboard`
4. On failure: shows the specific error message

Template: A centered Bulma card with 5 input fields (Full Name, Email, Username, Password, Confirm Password) and a "Create Account" button.

`client/src/views/DashboardView.vue`

The **main dashboard** at `/dashboard`. Shows the current user's fitness statistics.

Computed properties (auto-calculated from store data):

```
const userActivities = computed(() =>
  activitiesStore.getActivitiesByUser(authStore.currentUser.id)
)

const totalActivities = computed(() => userActivities.value.length)
const totalDuration   = computed(() => userActivities.value.reduce((sum, a) => sum +
a.duration, 0))
const totalCalories   = computed(() => userActivities.value.reduce((sum, a) => sum +
a.calories, 0))
const totalDistance   = computed(() => userActivities.value.reduce((sum, a) => sum +
(a.distance || 0), 0))
```

`.reduce()` is a JavaScript array method that accumulates a total. For example, `reduce((sum, a) => sum + a.duration, 0)` starts at 0 and adds each activity's duration to the running total.

Template: 4 "level-item" stat boxes side by side (Bulma's `level` layout):

- Total Activities (blue)
- Total Duration in minutes (info)
- Total Distance (green)
- Calories Burned (yellow)

These stats update **automatically** when new activities are added in `ActivitiesView`, because computed properties re-calculate whenever their dependencies change.

`client/src/views/ActivitiesView.vue`

The **activities management page** at `/activities`. Full CRUD (Create, Read, Update, Delete).

State:

- `isModalActive` — whether the add/edit modal popup is visible
- `isEditing` — whether the modal is in "edit" mode or "add" mode
- `editingId` — the ID of the activity being edited (null if adding)
- `form` — the reactive form data object (type, duration, distance, date, calories, notes)

CRUD Functions:

- **`openAddModal()`** — resets the form to empty defaults and opens the modal in "add" mode
- **`openEditModal(activity)`** — populates the form with existing activity data and opens in "edit" mode
- **`closeModal()`** — hides the modal
- **`saveActivity()`** — if editing: calls `updateActivity()`. If adding: calls `addActivity()` with `userId` set to the current user. Then closes the modal.
- **`deleteActivity(id)`** — shows a browser `confirm()` dialog, then calls `activitiesStore.deleteActivity()`

Template:

- A header row with the "Add Activity" button (top-right)
- A **Bulma table** displaying all user activities, sorted newest-first, with Edit and Delete buttons per row

- A **Bulma modal** with a form for adding/editing (date picker, activity type dropdown, duration, distance, calories, and notes textarea)

The modal's title (`'Edit Activity'` vs `'Add Activity'`) and the Save button's behavior are both controlled by `isEditing` .

> ***Key pattern**: The form is controlled by a single `form` ref object. Both the Add and Edit modal use the same form and modal — just with different initial data and different save logic.*

`client/src/views/FriendsView.vue`

The **friends page** at `/friends` . Read-only view of friends' activities.

State:

- `selectedFriend` — the currently selected friend (a `User` object or `null`)

Computed properties:

```
// Get the full User objects for each friend ID in currentUser.friends array
const friends = computed(() =>
  authStore.currentUser.friends
    .map(id => usersStore.getUserById(id))
    .filter((u): u is User => u !== undefined)  // Type-safe filter to remove undefined
)

// Get the selected friend's activities, sorted newest-first
const friendActivities = computed(() =>
  activitiesStore.getActivitiesByUser(selectedFriend.value.id)
    .sort((a, b) => new Date(b.date).getTime() - new Date(a.date).getTime())
)
```

Template: A 2-column layout:

- **Left column** (1/3 width): A Bulma `panel` listing friends. Clicking a friend sets `selectedFriend` .
- **Right column** (2/3 width): Shows the selected friend's activities in a table. If no friend is selected, shows a hint message.

> ***Important**: Friends can only **view** each other's activities — there are no Edit or Delete buttons here. This is intentional "read-only" access.*

`client/src/views/AdminView.vue`

The **admin panel** at `/admin` . Full CRUD for managing all users. Only accessible by admin users.

Extra security check in template:

```
<div v-if="!authStore.isAdmin" class="notification is-danger">
  You do not have permission...
</div>
<div v-else class="box"> ... admin content ... </div>
```

The router guard prevents non-admins from even navigating here, but this `v-if` adds a second layer of protection in case someone bypasses routing.

CRUD Functions:

- `openAddModal()` — opens modal in "add user" mode
- `openEditModal(user)` — populates form with existing user data
- `saveUser()` — calls `usersStore.updateUser()` or `usersStore.addUser()` as appropriate
- `deleteUser(id)` — prevents deleting yourself (checks `id !== currentUser.id`), then confirms and calls `usersStore.deleteUser()`

Template:

- Header with "Add User" button
- A table showing all users with their ID, avatar + name, username, email, role (color-coded tags), and Edit/Delete buttons
- The Delete button is **disabled** for the currently logged-in admin (can't delete yourself)
- A modal form for adding/editing users with Full Name, Username, Email, Password, and Role dropdown

---

`client/src/views/AboutView.vue`

A minimal placeholder "About" page. Static HTML, no logic. Publicly accessible.

🔄 How Everything Connects — Data Flow Diagram

```
index.html
    └── main.ts  (bootstraps Vue app)
            ├── Pinia  (global state)
            │     ├── usersStore     → array of User objects
            │     ├── authStore      → currentUser ref (uses usersStore)
            │     └── activitiesStore → array of Activity objects
            │
            ├── Vue Router  (navigation + guards)
            │     └── checks authStore.isLoggedIn / isAdmin before each route
            │
            └── App.vue  (root shell)
                    ├── NavBar.vue   (reads authStore → shows/hides links)
                    └── <RouterView> (renders one of the Views below)
                            ├── HomeView       (public)
                            ├── LoginView      (public → writes to authStore)
                            ├── RegisterView   (public → writes to authStore + usersStore)
                            ├── DashboardView  (auth → reads authStore + activitiesStore)
                            ├── ActivitiesView (auth → reads/writes activitiesStore)
                            ├── FriendsView    (auth → reads usersStore + activitiesStore)
                            └── AdminView      (admin → reads/writes usersStore)
```

🔑 Key Concepts to Master

1. `ref()` — Reactive Variables

```
const count = ref(0)
count.value++  // Must use .value to read/write in script
// In template: {{ count }} — no .value needed
```

2. `computed()` — Derived State

```
const doubled = computed(() => count.value * 2)
// Auto-updates whenever count changes. Read-only.
```

3. `v-model` — Two-Way Data Binding

```
<input v-model="username" />
<!-- Shorthand for: :value="username" @input="username = $event.target.value" -->
```

4. `v-if` / `v-else` — Conditional Rendering

```
<div v-if="authStore.isLoggedIn">Welcome!</div>
<div v-else>Please log in</div>
```

5. `v-for` — List Rendering

```
<tr v-for="activity in userActivities" :key="activity.id">
  <td>{{ activity.date }}</td>
</tr>
```

6. `defineStore()` — Pinia Store Pattern

```
export const useMyStore = defineStore('mystore', () => {
  const data = ref([])         // state
  const count = computed(...)  // getters
  function doSomething() {}    // actions
  return { data, count, doSomething }  // expose to components
})
```

7. `<script setup lang="ts">` — Vue Composition API

Modern Vue 3 syntax. Everything declared in `<script setup>` is automatically available in the template without `return {}`.

🎯 Demo Accounts (Quick Reference)

Role	Username	Password	Can access
Admin	`admin`	`admin123`	Everything including Admin panel

User	`john`	`john123`	Dashboard, Activities, Friends
Trainer	`jane`	`jane123`	Dashboard, Activities, Friends

⚠️ Important Limitations to Know

1. **Data is NOT persisted** — When you refresh the page, all changes (new activities, new users) are lost. There's no database or backend.
2. **Passwords are in plain text** — In a real app, passwords must be hashed. Never do this in production.
3. **No JWT tokens or sessions** — Auth state is stored only in Pinia (JavaScript memory). Refreshing logs you out.
4. **All data starts from seed** — The initial users and activities are hard-coded in the stores.

These are all acceptable for a course project, but would need fixing in a real-world application.